WEEK 7

Problem Statement:
Write a Java program to demonstrate different ways of creating and initializing `String` objects using various **String constructors**.

Code:

```
public class StringConstructor {
   public static void main(String[] args) {
String s1 = "Hello Java";
String s2 = new String("Hello Java");
char[] charArray = {'J', 'a', 'v', 'a'};
        String s3 = new String(charArray);
byte[] byteArray = {72, 101, 108, 108, 111};
        String s4 = new String(byteArray);
System.out.println("s1 (literal): " + s1);
        System.out.println("s2 (new String()): " + s2);
        System.out.println("s3 (from char array): " + s3);
        System.out.println("s4 (from byte array): " + s4);
        System.out.println("\ns1 == s2: " + (s1 == s2));
        System.out.println("s1.equals(s2): " + s1.equals(s2));
String s5 = "Hello Java";
        System.out.println("s1 == s5: " + (s1 == s5));
 }}
```

Output:

```
C:\Users\trivi>cd Javalab

C:\Users\trivi\Javalab>javac StringConstructor.java

C:\Users\trivi\Javalab>java StringConstructor.java
s1 (literal): Hello Java
s2 (new String()): Hello Java
s3 (from char array): Java
s4 (from byte array): Hello

s1 == s2: false
s1.equals(s2): true
s1 == s5: true

C:\Users\trivi\Javalab>
```

2) Problem Statement:

Write a Java program to demonstrate the functionality of the **StringBuffer** class by performing various mutable string operations.

Code:

```
public class StringBufferDemo {
    public static void main(String[] args) {
StringBuffer sb = new StringBuffer("Hello");
        System.out.println("Original: " + sb);
sb.append(" World");
        System.out.println("After append: " + sb);
sb.insert(5, " Java");
        System.out.println("After insert: " + sb);
sb.replace(0, 5, "Hi");
        System.out.println("After replace: " + sb);
sb.delete(0, 3);
        System.out.println("After delete: " + sb);
sb.reverse();
```

```
        System.out.println("After reverse: " + sb);
System.out.println("Length: " + sb.length());
        System.out.println("Capacity: " + sb.capacity());
    }}
```

Output:

```
C:\Users\trivi\Javalab>javac StringBufferDemo.java

C:\Users\trivi\Javalab>java StringBufferDemo.java
Original: Hello
After append: Hello World
After insert: Hello Java World
After replace: Hi Java World
After delete: Java World
After reverse: dlroW avaJ
Length: 10
Capacity: 21

C:\Users\trivi\Javalab>
```

3)Write a Java program to tokenize a given sentence using the **StringTokenizer** class and display each individual token.

CODE:

```
import java.util.Scanner;
import java.util.StringTokenizer;


public class StringTokenizerDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);


        System.out.print("Enter a sentence: ");
        String sentence = sc.nextLine();
StringTokenizer st = new StringTokenizer(sentence);
System.out.println("\n--- Tokens (default delimiter) ---");
        int count = 0;
        while (st.hasMoreTokens()) {
```

```
            System.out.println(st.nextToken());
            count++;
        }
        System.out.println("Total tokens: " + count);
System.out.print("\nEnter a custom delimiter (e.g. , or -): ");
        String delimiter = sc.nextLine();
StringTokenizer st2 = new StringTokenizer(sentence, delimiter);
System.out.println("\n--- Tokens (custom delimiter) ---");
        int count2 = 0;
        while (st2.hasMoreTokens()) {
            System.out.println(st2.nextToken());
            count2++;
        }
        System.out.println("Total tokens: " + count2);
    }
}
```

Output:

```
C:\Users\trivi\Javalab>javac StringTokenizerDemo.java

C:\Users\trivi\Javalab>java StringTokenizerDemo.java
Enter a sentence: Hi my name is Trivikram

--- Tokens (default delimiter) ---
Hi
my
name
is
Trivikram
Total tokens: 5

Enter a custom delimiter (e.g. , or -): .

--- Tokens (custom delimiter) ---
Hi my name is Trivikram
Total tokens: 1

C:\Users\trivi\Javalab>
```

4) **Problem Statement:**

Write a Java program to demonstrate **single inheritance** by creating a superclass containing common properties and methods and a subclass that inherits and extends the functionality of the superclass.

CODE:

```
class Person {
   String name;
   int age;
void display() {
      System.out.println("Name: " + name);
      System.out.println("Age: " + age);
   }}
class Student extends Person {
   int rollNo;
   String branch;
void displayStudent() {
      display();
      System.out.println("Roll Number: " + rollNo);
      System.out.println("Branch: " + branch);  }}
public class InheritanceDemo {
   public static void main(String[] args) {
      Student s1 = new Student();
      s1.name = "Rahul";
      s1.age = 20;
      s1.rollNo = 101;
      s1.branch = "CSE";s1.displayStudent();}}
```

Output:

```
C:\Users\trivi\Javalab>javac InheritanceDemo.java

C:\Users\trivi\Javalab>java InheritanceDemo.java
Name: Rahul
Age: 20
Roll Number: 101
Branch: CSE
```

5)Using super Keyword Problem Statement: Write a Java program to demonstrate the use of the super keyword for accessing superclass variables, methods, and constructors. Requirements: · Create a superclass Employee with appropriate variables and methods. · Create a subclass Manager. · Use super to access a superclass variable. · Use super to invoke a superclass method. · Use super() to invoke the superclass constructor. · Display the complete employee and manager details.

CODE:

```
class Employee {

   String name = "Unknown";

   double salary;

Employee() {

      System.out.println("Employee constructor called");

   }

Employee(String name, double salary) {

      this.name = name;

      this.salary = salary;

      System.out.println("Parameterized Employee constructor called");

   }

void display() {

      System.out.println("Name: " + name);

      System.out.println("Salary: " + salary);

   }}

class Manager extends Employee {
```

```
    String name = Manager Name
    double bonus;
Manager(String empName, double salary, double bonus) {
        super(empName, salary);
        this.bonus = bonus;
        System.out.println("Manager constructor called"); }
void displayManager() {
System.out.println("Superclass name (via super): " + super.name);
super.display();
System.out.println("Bonus: " + bonus); }}
public class SuperKeywordDemo {
    public static void main(String[] args) {
        Manager m1 = new Manager("Trivikram", 100000, 10000);
        System.out.println();
        m1.displayManager();}}
```

Output:

```
C:\Users\trivi\Javalab>javac SuperKeywordDemo.java

C:\Users\trivi\Javalab>java SuperKeywordDemo.java
Parameterized Employee constructor called
Manager constructor called

Superclass name (via super): Trivikram
Name: Trivikram
Salary: 100000.0
Bonus: 10000.0
```